

Student Registration App - Jetpack Compose + Room Database

Deskripsi Project

Student Registration App adalah aplikasi Android modern berbasis Kotlin dan Jetpack Compose yang menerapkan konsep CRUD (Create, Read, Update, Delete) menggunakan Room Database sebagai penyimpanan data lokal.

Aplikasi menggunakan:

- Kotlin
 - Jetpack Compose
 - Room Database
 - MVVM Architecture
 - Coroutine
 - Material Design 3
 - State Management Compose
-

Struktur Project

```
com.example.studentapp
├── data
│   ├── Siswa.kt
│   ├── SiswaDao.kt
│   └── AppDatabase.kt
├── ui
│   ├── MainScreen.kt
│   ├── StudentItem.kt
│   └── FormInput.kt
├── viewmodel
│   └── StudentViewModel.kt
└── MainActivity.kt
```

1. Dependency Gradle

build.gradle (Module)

```
plugins {  
    id("com.android.application")  
    id("org.jetbrains.kotlin.android")  
    id("kotlin-kapt")  
}  
  
android {  
    namespace = "com.example.studentapp"  
    compileSdk = 34  
  
    defaultConfig {  
        applicationId = "com.example.studentapp"  
        minSdk = 24  
        targetSdk = 34  
        versionCode = 1  
        versionName = "1.0"  
    }  
  
    buildFeatures {  
        compose = true  
    }  
  
    composeOptions {  
        kotlinCompilerExtensionVersion = "1.5.3"  
    }  
}  
  
dependencies {  
  
    implementation("androidx.core:core-ktx:1.12.0")  
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.6.2")  
    implementation("androidx.activity:activity-compose:1.8.0")  
  
    implementation(platform("androidx.compose:compose-bom:2023.10.01"))  
    implementation("androidx.compose.ui:ui")  
    implementation("androidx.compose.material3:material3")  
    implementation("androidx.compose.ui:ui-tooling-preview")  
  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.2")  
  
    implementation("androidx.room:room-runtime:2.6.1")  
    kapt("androidx.room:room-compiler:2.6.1")  
    implementation("androidx.room:room-ktx:2.6.1")  
}
```

```
        implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
    }
}
```

2. Entity

Siswa.kt

```
package com.example.studentapp.data
```

```
import androidx.room.Entity
import androidx.room.PrimaryKey
```

```
@Entity(tableName = "siswa")
data class Siswa(

    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,

    val nama: String,
    val email: String
)
```

3. DAO

SiswaDao.kt

```
package com.example.studentapp.data
```

```
import androidx.room.*
import kotlinx.coroutines.flow.Flow
```

```
@Dao
interface SiswaDao {

    @Insert
    suspend fun insertSiswa(siswa: Siswa)

    @Update
    suspend fun updateSiswa(siswa: Siswa)

    @Delete
    suspend fun deleteSiswa(siswa: Siswa)

    @Query("SELECT * FROM siswa ORDER BY id DESC")
```

```
        fun getAllSiswa(): Flow<List<Siswa>>
    }
}
```

4. Database Class

AppDatabase.kt

```
package com.example.studentapp.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(
    entities = [Siswa::class],
    version = 1
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun siswaDao(): SiswaDao

    companion object {

        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {

            return INSTANCE ?: synchronized(this) {

                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "siswa_database"
                ).build()

                INSTANCE = instance
                instance
            }
        }
    }
}
```

5. ViewModel

StudentViewModel.kt

```
package com.example.studentapp.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.studentapp.data.Siswa
import com.example.studentapp.data.SiswaDao
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch

class StudentViewModel(
    private val dao: SiswaDao
) : ViewModel() {

    val siswaList = dao.getAllSiswa()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(),
            initialValue = emptyList()
        )

    fun tambahSiswa(nama: String, email: String) {

        viewModelScope.launch {

            dao.insertSiswa(
                Siswa(
                    nama = nama,
                    email = email
                )
            )
        }
    }

    fun hapusSiswa(siswa: Siswa) {
        viewModelScope.launch {
            dao.deleteSiswa(siswa)
        }
    }

    fun editSiswa(siswa: Siswa) {
        viewModelScope.launch {
            dao.updateSiswa(siswa)
        }
    }
}
```

```
}  
}
```

6. Form Input

FormInput.kt

```
package com.example.studentapp.ui  
  
import androidx.compose.foundation.layout.*  
import androidx.compose.material3.*  
import androidx.compose.runtime.Composable  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.unit.dp  
  
@Composable  
fun FormInput(  
    nama: String,  
    email: String,  
    onNamaChange: (String) -> Unit,  
    onEmailChange: (String) -> Unit,  
    onTambahClick: () -> Unit  
) {  
  
    Column {  
  
        TextField(  
            value = nama,  
            onValueChange = onNamaChange,  
            label = {  
                Text("Nama")  
            },  
            modifier = Modifier.fillMaxWidth()  
        )  
  
        Spacer(modifier = Modifier.height(12.dp))  
  
        TextField(  
            value = email,  
            onValueChange = onEmailChange,  
            label = {  
                Text("Email")  
            },  
            modifier = Modifier.fillMaxWidth()  
        )  
  
        Spacer(modifier = Modifier.height(16.dp))  
    }  
}
```

```

        Button(
            onClick = onTambahClick,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Tambah Siswa")
        }
    }
}

```

7. Item Siswa

StudentItem.kt

```
package com.example.studentapp.ui
```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.studentapp.data.Siswa

```

```
@Composable
```

```

fun StudentItem(
    siswa: Siswa,
    onDelete: () -> Unit,
    onEdit: () -> Unit
) {

```

```

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp),
        shape = RoundedCornerShape(20.dp)
    ) {

```

```

Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
    verticalAlignment = Alignment.CenterVertically
) {

    Box(
        modifier = Modifier
            .size(50.dp)
            .clip(CircleShape)
            .background(Color(0xFFDCC8FF)),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = siswa.nama.take(1),
            fontSize = 20.sp
        )
    }

    Spacer(modifier = Modifier.width(12.dp))

    Column(
        modifier = Modifier.weight(1f)
    ) {

        Text(
            text = siswa.nama,
            style = MaterialTheme.typography.titleMedium
        )

        Text(
            text = siswa.email,
            style = MaterialTheme.typography.bodyMedium
        )
    }

    IconButton(onClick = onEdit) {
        Icon(Icons.Default.Edit, contentDescription = "Edit")
    }

    IconButton(onClick = onDelete) {
        Icon(Icons.Default.Delete, contentDescription = "Delete")
    }
}
}
}

```

8. Main Screen

MainScreen.kt

```
package com.example.studentapp.ui

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.studentapp.data.Siswa
import com.example.studentapp.viewmodel.StudentViewModel

@Composable
fun MainScreen(
    viewModel: StudentViewModel
) {

    var nama by remember {
        mutableStateOf("")
    }

    var email by remember {
        mutableStateOf("")
    }

    val siswaList by viewModel.siswaList.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {

        Text(
            text = "Registrasi Siswa",
            style = MaterialTheme.typography.headlineMedium
        )

        Text(
            text = "Kelola data siswa",
            style = MaterialTheme.typography.bodyLarge
        )

        Spacer(modifier = Modifier.height(20.dp))
    }
}
```

```

FormInput(
    nama = nama,
    email = email,
    onNamaChange = {
        nama = it
    },
    onEmailChange = {
        email = it
    },
    onTambahClick = {

        if (nama.isBlank()) {
            return@FormInput
        }

        if (email.isBlank()) {
            return@FormInput
        }

        if (!email.contains("@")) {
            return@FormInput
        }

        viewModel.tambahSiswa(nama, email)

        nama = ""
        email = ""
    }
)

Spacer(modifier = Modifier.height(20.dp))

if (siswaList.isEmpty()) {
    Text(
        text = "Belum ada data siswa"
    )
}

LazyColumn {
    items(siswaList) { siswa ->
        StudentItem(
            siswa = siswa,
            onDelete = {
                viewModel.hapusSiswa(siswa)
            }
        )
    }
}

```


Konsep State Management yang Digunakan

State

```
var nama by remember {  
    mutableStateOf("")  
}
```

remember

Digunakan untuk menyimpan state selama composable aktif.

mutableStateOf

Membuat state reactive.

collectAsState()

Mengubah Flow menjadi Compose State.

State Hoisting

State dipindahkan ke mainScreen lalu diteruskan ke FormInput.

Fitur Aplikasi

CRUD

- Tambah siswa
- Edit siswa
- Hapus siswa
- Tampilkan siswa

Validasi

- Nama wajib diisi
- Email wajib diisi
- Email harus valid

UI Modern

- Material 3
- Rounded corner
- Card modern
- Responsive layout

Database Lokal

- Room Database
 - SQLite
 - Flow reactive
-

Output Aplikasi

Aplikasi menampilkan:

- Form input nama
 - Form input email
 - Tombol tambah siswa
 - Daftar siswa
 - Tombol edit
 - Tombol hapus
 - Data tersimpan secara lokal
-

Kesimpulan

Project ini merupakan implementasi lengkap:

- Android modern development
- Jetpack Compose
- State Management
- Room Database
- MVVM Architecture
- CRUD Application
- Material Design 3